

1. Executive Summary & The Learner Problem

Low-Level Design (LLD) and Object-Oriented Design (OOD) are decisive evaluation criteria for software engineering roles from Mid-level to Staff level. Candidates are routinely asked to architect complex systems under ambiguous constraints (e.g., *Multi-Floor Parking Lot*, *Multi-Car Elevator Controller*, *Ride-Matching Service*).

Despite its industry importance, the current educational ecosystem fails learners due to four systemic deficiencies:

- **Passive Memorization vs. Active Design:** Learners consume static GitHub repositories ("awesome-low-level-design") or YouTube videos. These resources present a single person's solution, encouraging rote memorization of class diagrams rather than developing architectural trade-off intuition.
- **The "Single Canonical Solution" Fallacy:** Unlike LeetCode (where an algorithm has an exact input/output test oracle), **LLD has no single golden solution**. A Parking Lot can validly use a Strategy pattern for billing, a State pattern for spot occupancy, or a Specification pattern for vehicle assignment. Existing automated tools fail because they attempt brittle string matching against one reference answer.
- **The Immediate Feedback Void:** Self-study learners have no mechanism to evaluate whether their class hierarchies violate the Single Responsibility Principle (SRP), create circular inheritance, or introduce tight concrete coupling. Getting human critique requires expensive mock interviews with mentor scheduling friction.
- **Absence of an Iterative Learning Loop:** Professional software design is iterative: draft, critique, refactor, and improve. Current platforms treat design as a one-shot submission without quantifying how refactoring reduces coupling or improves cohesion across attempts.

2. Competitive Landscape & Analysis of Existing Approaches

An analysis of alternative platforms highlights why a dedicated LLD deliberate-practice workbench is missing:

Approach / Platform	Core Mechanism	Strengths	Critical Architectural Gaps
Static GitHub Repos & Blogs	Pre-written Java/C++ snippets & static UML images	Readily accessible, broad problem catalog, familiar patterns.	Zero interactivity ; no automated feedback; promotes copy-paste memorization of one specific approach.
Code Sandboxes (LeetCode, HackerRank)	Unit-test suite execution against code binaries	Automated grading, instant feedback, reliable execution.	Optimized for algorithms, not design ; ignores class relationships, inheritance depth, and trade-off justifications.
Pure LLM Chat (ChatGPT, Claude)	Free-form conversational prompting & chat replies	Flexible, capable of qualitative explanations and hints.	Non-deterministic & sycophantic ; praises flawed architectures; lacks deterministic graph linting; no version diffing or metrics.
Human Mock Platforms (Pramp, Interviewing)	Peer or mentor 1-on-1 video interviews	High-fidelity human feedback and nuanced conversation.	Expensive and non-scalable ; high scheduling friction; highly variable reviewer quality and evaluation rubrics.

Core Market Gap: Learners need an active modeling workbench that combines the *instant rigor of automated code grading* with the *nuanced critique of an architectural mentor*, without penalizing valid alternative designs.

3. Key Architectural Insights

Insight A: Hybrid Evaluation is Mandatory (Deterministic AST Linting + Semantic Heuristics)

Pure LLMs hallucinate rules and approve antipatterns. Pure static linters cannot evaluate semantic justification. The solution is a **two-stage hybrid pipeline**:

- **Deterministic Rule Engine (65% Weight):** Runs in $< 50 \text{ ms}$. Constructs an in-memory Abstract Syntax Tree (AST) and directed dependency graph. Deterministically validates cyclic inheritance (DFS back-edge detection), fan-in/fan-out efferent coupling, God classes (> 7 methods), and functional requirements mapping.
- **Semantic Reasoning Engine (35% Weight):** Evaluates domain entity coverage, design pattern justification, abstraction quality, and trade-off depth. Runs via a **Dynamic Heuristic Evaluator** locally, or connects to external LLMs with strict timeout and fallback guarantees.

Insight B: Structured Schema Modeling Outperforms Raw Boilerplate Code

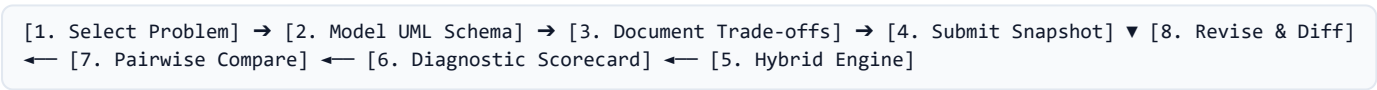
Requiring candidates to type 500 lines of Java boilerplate shifts cognitive focus to syntax and compiler typos rather than architecture. By providing a **structured design schema** (Classes, Interfaces, Methods, Attributes, Single Responsibilities, and UML Relationships), learners iterate rapidly. The system validates invariants instantly and renders dynamic visual UML via Mermaid.js.

Insight C: Learning Progression Occurs in the "Attempt-to-Attempt Diff"

Real learning occurs when a candidate sees: *Attempt 1 (God Class, 9 methods, score 41)* $\rightarrow$ *Feedback (Extract Pricing Strategy)* $\rightarrow$ *Attempt 2 (Strategy Pattern, score 94, +15 delta)*. Tracking attempt history and computing automated pairwise diffs (added interfaces, resolved violations, coupling reduction) provides the concrete reward loop of deliberate practice.

4. Product Direction: "LLD Arena"

LLD Arena is structured around a tight, 60-second deliberate practice loop:



Core Guiding Principles

1. **Zero Canonical Bias:** Grade against structural health, cohesion, and SOLID principles—never against a single "golden solution."
2. **Explainability Over Raw Scoring:** Every point deduction is linked to concrete evidence (e.g., *Class "ParkingLot" efferent coupling = 6; violates High Cohesion*) with actionable refactoring advice.
3. **Zero Infrastructure Friction:** Designed as a single-port TypeScript monolith with SQLite WAL. Runs 100% offline out-of-the-box without Docker, external databases, or mandatory API keys.

5. Scope & 2-Day Feasibility Justification

For a 2-day engineering assignment, disciplined scoping was paramount. We prioritized a rock-solid domain model, strict invariant enforcement, deterministic graph linting, and working pairwise attempt diffing over superficial distractions (e.g., multi-tenant OAuth, distributed queues, or complex WebAssembly sandboxes).